

HUB

AUTHOR
Version

Table of Contents

Table of contents

Class Index

Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

t_message_lcd (Structure pour la gestion de l'affichage de l'historique sur l'écran LCD)1

File Index

File List

Here is a list of all documented files with brief descriptions:

C:/Users/etulyon1/Documents/GitHub/iut-lyon1-but-geii-sae-6/Code/hub/Hub/src/images.h Erreur ! Signet non d
C:/Users/etulyon1/Documents/GitHub/iut-lyon1-but-geii-sae-6/Code/hub/Hub/src/main.cpp
(Programme principal pour le Hub de l'alarme)2

Class Documentation

t_message_lcd Struct Reference

Structure pour la gestion de l'affichage de l'historique sur l'écran LCD.

Public Attributes

- char **msg** [TRAME_SIZE]
- uint8_t **ligne**

Detailed Description

Structure pour la gestion de l'affichage de l'historique sur l'écran LCD.

Member Data Documentation

uint8_t t_message_lcd::ligne

Index de la ligne d'affichage

char t_message_lcd::msg[TRAME_SIZE]

Contenu textuel du message à afficher

C:/Users/etulyon1/Documents/GitHub/iut-lyon1-but-geii-sae-6/Code/hub/Hub/src/main.cpp File Reference

Programme principal pour le Hub de l'alarme.

```
#include <M5Core2.h>
#include <iostream>
#include <string>
#include <utility/In_eSPI.h>
#include <regex>
#include <Preferences.h>
#include <vector>
#include "images.h"
#include <FastLED.h>
```

Classes

struct **t_message_lcd** Structure pour la gestion de l'affichage de l'historique sur l'écran LCD.

Macros

- `#define LED_PIN 25`
- `#define NUM_LEDS 10`

Typedefs

- `typedef struct t_message_lcd t_message_lcd`
Structure pour la gestion de l'affichage de l'historique sur l'écran LCD.

Enumerations

- `enum Screen { HOME, OPTIONS, ALARM, PAIRING, KEYPAD, CHANGE_PWD, RFID_LIST }`
Énumération des différents écrans (Machine à états de l'interface graphique).
- `enum PwdChangeStep { ENTER_OLD, ENTER_NEW, CONFIRM_NEW }`

Functions

- `void Serial_callback ()`
Fonction de callback lors de la réception de données série.
- `String comparePwd (const char *PWD)`
- `String compareAdminPwd (const char *AdminPWD)`
- `String compareRfid (const char *RFID)`
- `void logSerial (const char *format,...)`
Formate et envoie une trace série vers le PC.
- `void taskGestionLcd (void *pvParameters)`
- `void drawAlarm ()`
- `void initAlarmButtons ()`
- `void drawHome ()`
Dessine l'écran d'accueil de l'interface.
- `void clearButtons ()`
Nettoie les objets Boutons de la mémoire.

- void **initHomeButtons** ()
- void **setScreen** (Screen s)
Effectue la transition vers un nouvel écran.
- void **initOptionsButtons** ()
- void **drawOptions** ()
- void **fadeLeds** (void)
Animation de "respiration" rouge sur la barre LED.
- bool **checkMACisInROM** (char *buffer)
- void **handleRFIDInput** (char *buffer, const String &IDhub, const String &IDdigi)
Traitement de la requête de vérification RFID entrante.
- void **handleAskLenghtMDP** (const String &IDhub, const String &IDdigi)
- void **handleAskStateAlarm** (const String &IDhub, const String &IDdigi)
- void **handleMDPInput** (char *buffer, const String &IDhub, const String &IDdigi)
Traite la requête de vérification de mot de passe (clavier).
- void **handleChangePwdKeyboard** (char *buffer)
- void **handlePairing** (char *buffer)
- void **taskTraiteTrame** (void *pvParameters)
Tâche : Analyse en continu des trames reçues sur le port série.
- void **taskTraiteSensor** (void *pvParameters)
Tâche : Gestion matérielle des capteurs et de la sirène.
- void **drawPairing** ()
- void **initPairingButtons** ()
- void **handlePairingLogic** ()
- void **BlinkLeds** (void)
Fait clignoter la barre LED en jaune.
- void **animateBtn** (int x, int y, int w, int h, int r, uint16_t c)
- void **initKeypadButtons** ()
- void **drawKeypad** ()
- void **updateKeypadHeader** ()
- void **handleKeypadLogic** ()
- void **handleAlarmLogic** ()
- void **drawChangePwd** ()
- void **updateChangePwdHeader** ()
- void **handleChangePwdLogic** ()
- void **loadRFIDBadges** ()
Charge les badges RFID depuis la mémoire persistante.
- void **saveRFIDBadges** ()
Sauvegarde les badges RFID dans la mémoire persistante.
- void **drawRfidList** ()
- void **initRfidButtons** ()
- void **handleRfidListLogic** ()
- void **handlePassAdminInput** (char *buffer)
- void **handleAddRFID** (char *buffer)
- String **getMacFactory** (void)

Récupère l'adresse MAC usine.

- void **setup** ()
Fonction d'initialisation (Setup).
- void **handleStateTransitions** ()
- void **handleHomeLogic** ()
- void **handleOptionsLogic** ()
- void **loop** ()
Boucle principale du programme.
- void **updatePairingStatus** ()

Variables

- constexpr uint16_t **C_BG** = TFT_BLACK
- constexpr uint16_t **C_YELLOW** = TFT_YELLOW
- constexpr uint16_t **C_RED** = TFT_RED
- constexpr uint16_t **C_ORANGE** = 0xE8E4
- constexpr uint16_t **C_DARKGREEN** = 0x0546
- constexpr uint16_t **C_BLUE** = 0x039F
- constexpr uint16_t **C_GREY** = 0x7BEF
- constexpr uint16_t **C_WHITE** = TFT_WHITE
- constexpr uint16_t **C_OPT** = 0x18E3
- String **keyboardPwd** = "1111"
- String **PassAdmin** = "1234"
- std::vector< String > **rfidBadges**
- String **IDdigi** = "0000"
- String **IDhub** = "00000000000000"
- uint8_t **sensor1** = 19
- uint8_t **alarmSiren** = 27
- uint8_t **faceDetector1** = 32
- uint8_t **faceDetector2** = 33
- bool **RFIDEnabled** = false
- CRGB **leds** [NUM_LEDS]
- constexpr int **TRAME_SIZE** = 128
- constexpr int **Ecart_Text** = 20
- constexpr int **W** = 320
- constexpr int **H** = 240
- constexpr int **HEADER_H** = 50
- constexpr int **MGN** = 10
- constexpr int **GAP** = 10
- constexpr int **BTN_W** = (W - (MGN * 2) - GAP) / 2
- constexpr int **R1_Y** = 90
- constexpr int **R1_H** = 85
- constexpr int **R1_R** = 15
- constexpr int **R2_Y** = 180
- constexpr int **R2_H** = 50
- constexpr int **R2_R** = 10
- constexpr int **C1_X** = MGN
- constexpr int **C2_X** = MGN + BTN_W + GAP
- constexpr int **O_W** = BTN_W
- constexpr int **O_H** = 50
- constexpr int **O_R** = 8
- constexpr int **O_GAP_Y** = 10
- constexpr int **O_Y1** = 60
- constexpr int **O_Y2** = O_Y1 + O_H + O_GAP_Y
- constexpr int **O_Y3** = O_Y2 + O_H + O_GAP_Y

- `constexpr int OPTION_COUNT = 6`
- `int optX [OPTION_COUNT] = {C1_X, C2_X, C1_X, C2_X, C1_X, C2_X}`
- `int optY [OPTION_COUNT] = {O_Y1, O_Y1, O_Y2, O_Y2, O_Y3, O_Y3}`
- `const char * optNames [OPTION_COUNT] = {"O1", "O2", "O3", "O4", "O5", "O6"}`
- `const char * optLabels [OPTION_COUNT] = {"MDP Clavier", "MDP Admin", "Ecran", "Appairage", "Badges RFID", "Retour"}`
- `constexpr int KP_BTN_W = 80`
- `constexpr int KP_BTN_H = 35`
- `constexpr int KP_GAP_X = 20`
- `constexpr int KP_GAP_Y = 10`
- `constexpr int KP_START_X = 20`
- `constexpr int KP_START_Y = 60`
- `String enteredPin = ""`
- `Screen previousScreenForKeypad = HOME`
- `Button * bKp [12] = {nullptr}`
- `PwdChangeStep pwdStep = ENTER_OLD`
- `String tempNewPwd = ""`
- `bool isAdminPwdChange = false`
- `Button * bRfidDel [4] = {nullptr}`
- `Button * bRfidPrev = nullptr`
- `Button * bRfidNext = nullptr`
- `Button * bRfidRet = nullptr`
- `int rfidPage = 0`
- `Screen currentScreen = HOME`
- `bool locked = false`
- `bool isPairingMode = false`
- `Preferences preferences`
- `Button * bLock = nullptr`
- `Button * bBlue = nullptr`
- `Button * bOpts [OPTION_COUNT] = {nullptr}`
- `SemaphoreHandle_t lcdMutex = nullptr`
- `char displayBuffer [4][TRAME_SIZE]`

Detailed Description

Programme principal pour le Hub de l'alarme.

Gère l'interface utilisateur tactile, la communication série avec le digicode, et les capteurs matériels (mouvement, détection faciale, sirène, etc.)

Enumeration Type Documentation

enum Screen

Énumération des différents écrans (Machine à états de l'interface graphique).

Enumerator:

HOME	Écran d'accueil principal (Logs et statuts)
OPTIONS	Menu des paramètres (Mot de passe, Appairage, etc.)
ALARM	Écran d'alerte lors d'une intrusion
PAIRING	Interface pour autoriser un nouveau périphérique

KEYPAD	Pavé numérique pour déverrouiller l'alarme
CHANGE_PWD	Interface de modification du code PIN
RFID_LIST	Base de données des badges RFID

Function Documentation

void BlinkLeds (void)

Fait clignoter la barre LED en jaune.

Indique une intrusion

void clearButtons ()

Nettoie les objets Boutons de la mémoire.

Supprime tous les boutons avant chaque transition d'écran

void drawHome ()

Dessine l'écran d'accueil de l'interface.

Gère l'affichage du statut de l'alarme, de l'historique et des boutons principaux

void fadeLeds (void)

Animation de "respiration" rouge sur la barre LED.

Indique que l'alarme est armée

String getMacFactory (void)

Récupère l'adresse MAC usine.

Returns

L'adresse MAC sous forme de chaîne de caractères

void handleMDPInput (char * buffer, const String & IDhub, const String & IDdigi)

Traite la requête de vérification de mot de passe (clavier).

Parameters

<i>buffer</i>	Trame brute reçue
<i>IDhub</i>	Adresse MAC locale du Hub
<i>IDdigi</i>	Adresse MAC du digicode distant

void handleRFIDInput (char * buffer, const String & IDhub, const String & IDdigi)

Traitement de la requête de vérification RFID entrante.

Parameters

<i>buffer</i>	La trame brute reçue par la communication série
<i>IDhub</i>	L'adresse MAC locale du Hub
<i>IDdigi</i>	L'adresse MAC du digicode distant

void loadRFIDBadges ()

Charge les badges RFID depuis la mémoire persistante.

Transforme la chaîne stockée en un vecteur de chaînes de caractères

void loop ()

Boucle principale du programme.

Gère les interactions tactiles et allume la barre LED

void saveRFIDBadges ()

Sauvegarde les badges RFID dans la mémoire persistante.

Convertit le vecteur de badges en une chaîne compacte séparée par des ','

void Serial_callback ()

Fonction de callback lors de la réception de données série.

Lit les caractères sur le port série 2 et les ajoute à la queue réception.

void setScreen (Screen s)

Effectue la transition vers un nouvel écran.

Parameters

<i>s</i>	L'écran cible à afficher
----------	--------------------------

void setup ()

Fonction d'initialisation (Setup).

Configure l'écran du M5, le port I2C/Série, les LEDs et alloue les tâches

void taskTraiteSensor (void * pvParameters)

Tâche : Gestion matérielle des capteurs et de la sirène.

Gère les états logiques des PINs du capteur de mouvement, du relais de la sirène et de la caméras)

void taskTraiteTrame (void * pvParameters)

Tâche : Analyse en continu des trames reçues sur le port série.

Lit uniquement les trames reconnus

Index

INDEX